Exam 1 - Results

Attempt 1 of 1

Written Oct 9, 2024 11:21 AM - Oct 9, 2024 11:34 AM

Attempt Score **73.33 %**

Overall Grade (Highest Attempt) **73.33 %**

Question 1 **5 / 5 points**

Suppose we created a **class** in Swift called Point, with properties x and y, and the usual initializer. Is there any way that we could follow the declaration

```
let p = Point(1, 2)
```

with

```
p.x = 5
```

- ( ) No, let-constants make the instances they refer to immutable. We would have to change let to var to make the instance mutable.
- ( ) Yes, as long as the property x is computed, rather than stored.
- ( ) No, because the declaration is wrong. The `new` operator is missing.
- ✓ ( ) Yes, the let only means we can't change what p is bound to, but we may still be able to mutate the properties of the instance.
- ( ) No, properties are private by default in Swift.

Question 2 **5 / 5 points**

What are the cases of a Swift optional called?

- ✓ ( ) some and none

- ◯ of and empty
- ◯ success and failure
- ◯ of and nil

Question 3 **1 / 5 points**

Here is a Java statement

```
switch (color) {
  case Color.RED: System.out.println("stop");
  case Color.YELLOW: System.out.println("slow down");
  case Color.GREEN: System.out.println("go");
}
```

Color is an enum with three variants. This code will actually run, but it doesn't do what is intended. Which of the following fix the problem?

- ☐ ✗
```
var action = switch (color) {
  case Color.RED -> "stop";
  case Color.YELLOW -> "slow down";
  case Color.GREEN -> "go";
}
System.out.println(action);
```
- ☐ ✗
```
String action = switch (color) {
  case Color.RED -> "stop";
  case Color.YELLOW -> "slow down";
  case Color.GREEN -> "go";
}
```
- ☐ ✗
```
switch (color) {
  case Color.RED -> System.out.println("stop");
  case Color.YELLOW -> System.out.println("slow down");
  default -> System.out.println("go");
}
```
- ☐ ✗
```
System.out.println(switch (color) {
  case Color.RED -> "stop";
  case Color.YELLOW -> "slow down";
  case Color.GREEN -> "go";
});
```
- ☑
```
when (color) {
  | Color.RED: System.out.println("stop");
  | Color.YELLOW: System.out.println("slow down");
  | Color:GREEN: System.out.println("go");
}
```

Question 4 **5 / 5 points**

In JavaScript 3**35 = 50031545098999710 but in Python 3**35 = 50031545098999707. Which of the following statements is false?

- ◯ JavaScript's answer is inaccurate because it uses floating-point math
- ◯ Python's answer is spot on accurate and JavaScript's is not
- ✓ ◯ For unsafe integers, JavaScript always pads out its computations with zeros
- ◯ JavaScript would have gotten the exact numeric (mathematical) value had we written `3n**35n`

Question 5 **3 / 5 points**

Which are true regarding expressions and statements?

- ➡ ✓ ☐ Statements do not produce values, but rather produce actions or effects
- ➡ ✓ ☐ Expressions produce (evaulate to) values
- ✓ ☐ Functions can only have statements, not expressions
- ✗ ☐ Python only allows assignment in assignment statements
- ➡ ✗ ☐ Both can manipulate values

Question 6 **5 / 5 points**

Which of the following are good technical reasons for Swift not having sealed classes?

- ✓ ☐ Python doesn't have them, so not having them is good enough for Swift
- ✓ ☐ Adding them to the language would be admitting the Android people did a better job designing language features so the Swift community would rather pretend they don't exist
- ✓ ☐ Enums are so powerful and flexible that they do everything sealed classes would do
- ✓ ☐ People overuse them in practice and Swift doesn't want people tempted to overuse them

Question 7 **0 / 5 points**

Recall that `x?.y` is the safe way to look up property `y` from `x` in JavaScript: if `x` is nullish you get undefined, otherwise you (safely) get `x.y`. Let's try to capture the similar idea in Python, where we want to look up `x['y']` but we think that `x` might

be `None` and we don't want our program to crash; instead we just want the whole expression to be `None`. What is the closest Python equivalent to the JavaScript expression `x?.y` ?

- ✗ ◯ `x.get("y", None)`
- ◯ `x or y`
- ◯ `x?.y`
- ➡ ◯ `(x or {}).get("y")`

Question 8 **5 / 5 points**

Since Swift structures have value semantics, how should we typically design functions that accept structures as parameters?

- ◯ We should mark the parameters var to avoid copies
- ✓ ◯ Don't do anything special, pass them normally. Parameters can't be written to, anyway, and Swift is copy-on-write, so no copies are made. We should only make the parameter inout if it is to be modified, which should be done sparingly anyway.
- ◯ We should cast them to classes to avoid copies
- ◯ They should always be passed as inout so that we get efficiency due to having only pointers passed at runtime

Question 9 **5 / 5 points**

Here is a JavaScript generator

```
function* constantStepper(start, step) {
  let element = start
  while (true) {
    yield element
    element += step
  }
}
```

What is the closest Python translation of it?

```
def constant_stepper(start, step):
  local element = start
  while (True):
    yield element
    element = element + step
```

```
def* constant_stepper(start, step):
  let element = start
  while True:
    yield element
    element += step
```

✔
```
def constant_stepper(start, step):
  element = start
  while True:
    yield element
    element += step
```

```
def* constant_stepper(start: int, step: int):
  var element = start
  while True:
    yield element
    element += step
```

Question 10 **5 / 5 points**

What is the equivalent of JavaScript's `x?.y` in Java (assuming `x` refers to an optional that wraps an object with a field called `y`)?

✔ `x.map(x -> x.y)`

`x != Optional.empty() ? Optional.empty() : x.get().y`

`x.orElse(x.y)`

`x == null ? null : x.y`

Question 11 **4 / 5 points**

Match up the languages with the expression they use to compute the sign of a number

- ✓ __1__ Kotlin
- ✗ __1__ (2) Swift
- ✓ __2__ JavaScript
- ✓ __3__ Python
- ✓ __2__ Java

1. `if (x < 0) -1 else if (x > 0) 1 else 0`
2. `x < 0 ? -1 : x > 0 ? 1 : 0`
3. `-1 if x < 0 else 1 if x > 0 else 0`

Question 12 **4 / 5 points**

Which of the following are generated for you when defining a Java record?

- ✓ ☐ Getters and setters
- ➡ ✓ ☐ a toString method
- ✗ ☐ a componentN method
- ➡ ✓ ☐ Private fields
- ➡ ✓ ☐ equals and hash code methods

Question 13 **5 / 5 points**

Which of the following are atomic, as opposed to composite, values

- ✓ ☐ character
- ✓ ☐ integer
- ✓ ☐ reference
- ✓ ☐ symbol
- ✓ ☐ string
- ✓ ☐ quaternion

Question 14 **2 / 5 points**

Which of the following are true statements regarding literals and variables?

- ✓ ☐ Both are values
- ➡ ✗ ☐ Both are entities

- ✗ ☐ Variables contain values but literals *are* values
- ➡ ✗ ☐ Variable contain values but literals represent, or denote, values
- ➡ ✓ ☐ Variables can be named but literals cannot

Question 15 **5 / 5 points**

Which is the best JavaScript translation of the Python function declaration

```
def draw(x, y, /, *, color, width)
```

- ○ `function draw(x, y, {ofColor: color, withWidth: width})`
- ○ `function draw([x, y], {color, width})`
- ✓ ○ `function draw(x, y, {color, width})`
- ○ `function draw({x, y, color: color, width: width})`
- ○ `function draw({x, y}, {color, width})`

Question 16 **5 / 5 points**

What is the Python equivalent of JavaScript's `{x: 3, [y]: 5, z}` ?

- ○ `make_dictionary(z=z, y=5, x=3}`
- ○ `{y: 5, z, x: 3}`
- ✓ ○ `{y: 5, "z": z, "x": 3}`
- ○ `{"z":z, x:3, :"y": 5}`

Question 17 **2 / 5 points**

Which of the following aspects of Java classes were know to, and rejected by, designers of both Swift and Kotlin?

- ➡ ✓ ☐ Allowing null to be assigned to any variable constrained to any class type
- ✗ ☐ Static fields and methods
- ✓ ☐ Overloading of constructors
- ➡ ✗ ☐ Requiring a class definition even when only a single object of that class must ever exist

- [ ] Disallowing functions outside of classes

Question 18 **3.333 / 5 points**

Which of the following are accurate statements about records and dictionaries?

- [x] Dictionaries are essentially records with labeled components
- [x] In mainstream programming languages, records use parentheses while dictionaries use curly braces
- [x] Both can be considered to be collections of pairs
- [ ] Dictionaries are intended to show the value of a single property over many different objects, while a record is intended to show the properties of a single object
- [ ] Records are immutable but dictionaries are mutable
- [x] There's no difference: these are just two names for a single concept

Question 19 **4 / 5 points**

In Unicode, "woman scientist" is considered

- [x] An incomplete character, because no skin tone modifier is present
- [x] A grapheme
- [x] A code point
- [ ] An emoji
- [x] A character

Question 20 **0 / 5 points**

What is the *most common* way to create a binding in mainstream languages?

- ( ) Pattern Match
- ( ) Declaration
- ( ) Return statement
- ( ) Assignment

Done